Figure 3.2 In a DNS amplification attack, the attacker sends the same DNS query to many DNS servers, spoofing their IP address to look like the request came from the victim. By carefully choosing the DNS query, the server can be tricked into replying with much more data than was in the original query, flooding the victim with traffic.

As mentioned in chapter 1, once a user is logged in, you can apply *quotas* to restrict the resources that each user may consume. For example, we could restrict the number of social spaces that each user can create or limit the number of messages they can send each day. This is not always possible, and even unauthenticated requests will consume some resources. As we will see in the next section, authenticating a request can itself be an expensive operation. Before a user has authenticated, we still need a way to prevent large spikes in traffic from a small number of clients from preventing access to the API to legitimate clients. If the API is overwhelmed with requests, then it may eventually crash, preventing access to anybody and potentially also losing data. A basic defense against DoS attacks is to apply *rate-limiting* to all requests, ensuring that we never attempt to process more requests than our server can handle. It is better to reject some requests in this case, than to crash trying to process everything. Genuine clients can retry their requests later when the system has returned to normal.

Rate-limiting should be the very first security decision that you make when a request reaches your API. Because the goal of rate-limiting is ensuring that your API has enough